المدرسة الوطنية للعلوم التطبيقية - بني ملال
Ecole Nationale des Sciences Appliquées - Béni Mellal

Rapport de Travaux Pratiques

TP1 -- Application Monolithique avec Spring Boot

Realisee par	Nawar El Asraoui
Filiere	Cybersecurite et Intelligence Artificielle
Encadre par	Pr. BE ELBAGHAZAOUI
Annee universitaire	2025 -- 2026

Mars 2026

1. Introduction et Objectifs

Ce TP a pour objectif de creer une **application Spring Boot monolithique** permettant la gestion d'un systeme e-commerce simplifie. L'application implemente une API REST complete couvrant quatre modules : **Produits, Categories, Clients et Commandes**. L'architecture en couches classique (Model, Repository, Service, Controller, DTO, Mapper) est mise en oeuvre avec une base de donnees embarquee H2 en memoire, accessible sur le port 8081.

1.1 Objectifs pedagogiques

- Creer une application Spring Boot monolithique complete
- Developper une API REST CRUD pour plusieurs entites liees
- Comprendre l'architecture en couches et le pattern DTO
- Implementer les relations JPA entre entites : **ManyToOne (Product -> Category)** et **OneToMany (Order -> OrderItem)**
- Utiliser la base de donnees H2 embarquee pour les tests sans infrastructure externe
- Identifier les avantages et limites d'une architecture monolithique

1.2 Technologies utilisees

Technologie	Role
Spring Boot 4.0	Framework principal de l'application
Spring Data JPA	Persistance et acces aux donnees
Base de donnees H2	BDD embarquee en memoire (port 8081)
Lombok	Reduction du code boilerplate (getters, constructeurs)
MapStruct	Mapping automatique entre entites et DTOs
Jakarta Validation	Validation des donnees entrantes (@NotBlank, @Positive...)
Maven	Gestion des dependances et compilation
Postman	Tests des endpoints REST

2. Architecture du Projet

L'application est organisee selon une **architecture en couches**, decoupee par domaine fonctionnel. Chaque module (product, customer, order) possede ses propres sous-packages : model, dto, mapper, repository, service et controller. Cette organisation garantit une separation claire des responsabilites et facilite la maintenance du code.

2.1 Modele de donnees

Le systeme est compose de cinq entites JPA persistees dans la base H2 :

Entite	Table BDD	Champs principaux
Product	products	id, name, description, price, stock, category_id
Category	categories	id, name (unique)
Customer	customers	id, fullName, email (unique)
Order	orders	id, customerId, createdAt, total, items
OrderItem	order_items	id, order_id, productId, productName, quantity, unitPrice

2.2 Relations entre entites

Deux types de relations JPA sont implementes : **Product -> Category (ManyToOne)** : un produit appartient a une categorie. La cle etrangere **category_id** est stockee dans la table products avec un chargement lazy (FetchType.LAZY) pour optimiser les performances. **Order -> OrderItem (OneToMany)** : une commande contient plusieurs lignes d'articles, avec cascade ALL et orphanRemoval pour la gestion automatique du cycle de vie.

2.3 Pattern DTO et Mapper

L'application utilise le pattern DTO (Data Transfer Object) pour decoupler la couche HTTP de la couche persistance. MapStruct genere automatiquement les classes de conversion. Ainsi, les donnees exposees par l'API sont distinctes des entites JPA, ce qui protege le schema interne de la base de donnees et permet de personaliser les reponses (ex : inclure le nom de la categorie dans ProductDTO).

3. Presentation des Modules

3.1 Module Produits

Le module Produits est le coeur de l'application. Il gere le catalogue de produits avec leurs prix, stocks et categories associees. Le service ProductService centralise la logique metier : lors de la creation ou de la modification d'un produit, il verifie l'existence de la categorie demandee avant de l'associer. En cas d'identifiant de categorie invalide, une exception IllegalArgumentException est levee avec un message explicite. Le ProductDTO retourne en reponse inclut non seulement l'identifiant de la categorie mais aussi son nom, evitant ainsi un second appel API cote client.

3.2 Module Categories

Le module Categories offre deux endpoints simples : lister toutes les categories et en creer une nouvelle. La contrainte d'unicite sur le nom (unique = true) est appliquee au niveau JPA et base de donnees, garantissant l'integrite des donnees. Une categorie est creee avant les produits qui lui sont rattaches.

3.3 Module Clients

Le module Customer gere les informations des clients de la plateforme. L'entite Customer contient les champs **fullName** et **email** (unique). Le CustomerDTO expose ces informations via l'API sans exposer les details internes de l'entite JPA. Le service implemente la creation et la consultation de clients, avec verification de l'existence par identifiant.

3.4 Module Commandes

Le module Order est le plus complexe : il implique une relation entre clients, produits et lignes de commande. Une commande (Order) reference un client par son identifiant et contient une liste d'OrderItem, chacun referenceant un produit avec sa quantite et son prix unitaire au moment de la commande. Le total de la commande est calcule automatiquement. La route GET /api/customers/{id}/orders retourne l'historique complet des commandes d'un client, suivant les conventions RESTful d'URL imbriquee.

4. Endpoints REST

L'ensemble des endpoints exposes par l'application est recapitule dans le tableau suivant. L'application ecoute sur le port 8081 et la console H2 est accessible a l'adresse http://localhost:8081/h2-console.

Methode	URL	Description
GET	/api/products	Lister tous les produits
GET	/api/products/{id}	Obtenir un produit par ID
POST	/api/products	Creer un produit
PUT	/api/products/{id}	Modifier un produit
DELETE	/api/products/{id}	Supprimer un produit
GET	/api/categories	Lister les categories
POST	/api/categories	Creer une categorie
GET	/api/customers	Lister les clients
GET	/api/customers/{id}	Obtenir un client par ID
POST	/api/customers	Creer un client
POST	/api/orders	Creer une commande

GET	/api/customers/{id}/orders	Historique des commandes client

5. Avantages et Limites du Monolithe

5.1 Avantages observes

- **Simplicite de developpement :** un seul projet, un seul deploiement, une seule base de donnees. Le demarrage de l'application est immediat avec Spring Boot.
- **Coherence transactionnelle :** toutes les operations entre modules partagent le meme contexte de transaction JPA, garantissant la coherence des donnees.
- **Facilite de test :** toutes les couches sont accessibles dans le meme contexte Spring, ce qui simplifie les tests d'integration.
- **Performances intra-application :** les appels entre services se font directement en memoire, sans latence reseau entre modules.

5.2 Limites identifiees

- **Scalabilite limitee :** impossible de scaler uniquement le module Commandes sans redéployer l'ensemble de l'application.
- **Couplage fort :** une modification dans le module Produits peut impacter les modules Commandes et Clients.
- **Deploiement risque :** tout changement, meme mineur, necessite de recompiler et redeployer l'application entiere.
- **Base de donnees unique :** tous les modules partagent la meme instance H2, creant un point de defaillance unique.

6. Conclusion

Ce TP a permis de concevoir et d'implementer une application monolithique complete avec Spring Boot, couvrant quatre modules metier interconnectes. L'architecture en couches avec le pattern DTO a permis d'organiser le code de facon claire et maintenable. La relation ManyToOne entre Product et Category et la relation OneToMany entre Order et OrderItem ont illustre la gestion des associations JPA. La base de donnees H2 embarquee a facilite les tests sans infrastructure externe. Ces fondations monolithiques serviront de base de comparaison pour les architectures distribuees et microservices abordees dans les TPs suivants du module.